CSE 213 — Software Engineering

Software Testing — Types, Coverage, Equivalence Partitioning and BVA

Lesson notes · Exam approach · TF practice with solutions · Quick notes (Anindya Sir's portion)

How to Read These Notes

EXAM FOCUS	Exam importance, where it appears in the TF collection
MEMORIZE	Rules, tables and formulas to memorize
TRAP	Common mistakes and exam traps
EXAM-READY ANSWER	Model wording you can write in the exam
NOTE	Extra explanation / clarification
Grey box, monospace	Code, worked tables, derivations and hand-drawn sketches
Centered grey box	Formula to remember / final numerical result
Purple heading / ✓ green	Practice questions / their solutions

Contents

Exam Map — What the TF Papers Ask — testing questions — the most frequent topic of this section
Lesson 23 — Testing Basics: V&V, Fault Terms, Black Box vs White Box — Lec 11 sl. 1–13
Lesson 24 — Six Types of Testing, Integration Order, Performance Tests — Lec 11 sl. 14–22
Lesson 25 — Code Coverage Criteria — 1 - Coverage.pptx sl. 1–8
Lesson 26 — White-Box Test Design: Basis Paths and Loop Testing — textbook (asked often)
Lesson 27 — Equivalence Partitioning, BVA, Positive and Negative Testing — 2 - Equivalence partitioning.pptx sl. 1–14
Quick Notes — Software Testing — revision

NOTE Slide numbers refer to the PDF page numbers of the deck named under each lesson title. TF page numbers refer to 'CSE 213 All Past Papers.pdf' (70 pages); the newest paper (22 batch, labelled 2024-25 here) is a separate file, cited as '22 batch p.N'. Old course code: CSE 307. Lessons 23–24: 'Lec 11 - Software testing.pdf' (22 slides). Lesson 25: '1 - Coverage.pptx' (8 slides). Lesson 27: '2 - Equivalence partitioning, positive and negative testing.pptx' (14 slides). Lesson 26 (cyclomatic complexity, basis paths, loop testing) is textbook material (Pressman ch. 23) needed for the white-box questions; it is marked 'textbook'.

Exam Map — What the TF Papers Ask

Session (paper)	Questions in this note file
2024-25 (CSE 213)	Q5(a) Course registration ECT (credits C1–C4, CGPA G1–G4, status S1–S3): minimal weak ECT, strong count, defect missed by weak (15) [L27] · Q6(c) Integration testing Display–Controller(missing)–Sensor: stubs/drivers, order (10) [L24] · Q7(a) Performance test plan: burst 10,000 users, collapse beyond capacity, slowdown after 8 h — test types + JMeter configuration (15) [L24] · Q7(b) BVA for the LOW partition 30 ≤ M ≤ 70 (10) [L27]
2023-24 (CSE 213)	Q1(c) Design test cases for the password policy (10) [L27] · Q3(b) White-box test cases for FindMean (10) [L26] · Q3(c) Equivalence partitioning principle; why it is used (8) [L27] · Q5(c) Why high test coverage is important; risks of low coverage (8) [L25]
2022-23 (CSE 307)	Q1(b) Test cases for income-tax slabs 0 % < 4 lac, 10 % 4–10 lac, 20 % > 10 lac (10) [L27] · Q1(c) Internal and external integration testing for a ride-sharing app (10) [L24] · Q4(c) something(int x): if (x < 0 || y < x) — minimal data set for condition coverage (8) [L25]
2021-22 (CSE 307)	Q3(b) Six testing types: unit, integration, non-functional, regression, acceptance, beta (10) [L24] · Q3(c) 'Less cost to fix a bug found later in the SDLC' — agree? (5) [L23] · Q4(c) Five negative-testing ideas for a web application (5) [L27]
2020-21 (CSE 307)	Q5(b) Equivalence partitioning; test input set for Release Order cost (12) [L27] · Q6(b) Test execution ensuring method, branch and condition coverage; characteristics of good unit tests (15) [L25] · Q7(a) GM laptop + two phones per year requisition: positive and negative testing (10) [L27] · Q8(c) Smoke vs regression testing with example (8) [L24]
2019-20 (CSE 307)	Q5(b) ERP HRM: 4 inter-component + 2 inter-module integration-test requirements (15) [L24] · Q8(b) Load vs stress testing for BIIS; is code review black-box testing? (15) [L24]
2018-19 (CSE 307)	Q1(b) Equivalence partitioning; test input set for BIIS user roles (12) [L27] · Q2(b) Test execution ensuring method, branch and condition coverage; good unit testing (15) [L25] · Q3(a) GM laptop requisition: positive and negative testing (7) [L27] · Q4(a) Integration, beta and usability testing (9) [L24]
2017-18 (CSE 307)	Q7(a) Spec 100 users; failed at 90, crashed at 150 — which two test types? (6) [L24] · Q8(a) Black-box advantages/disadvantages; performance and regression testing (12) [L23]
2014-15 (CSE 307)	Q2(a) Example showing exhaustive testing is nearly impossible (6⅔) [L23] · Q2(b) Loop-testing test cases: max of n numbers (12) [L26] · Q2(d) Components of the unit-test environment (7) [L24] · Q2(e) Integration-testing strategies (8) [L24]
2013-14 (CSE 307)	Q3(a) Call-return tree 1–14: top-down integration order with stubs and drivers (10) [L24] · Q3(b) Debugging process with diagram (8) [L23] · Q3(d) Loop-testing test cases: sum of n numbers (10) [L26] · Q3(e) Equivalence partitioning to reduce test cases — example (6⅔) [L27]
2012-13 (CSE 307)	Q4(c) Counting sort: flow graph, independent paths, test cases (20) [L26] · Q4(d) Verification vs validation; defect testing vs debugging (6) [L23]
2011-12 (CSE 307)	Q7(a) Stock trading: unit, integration or system test for five cases (10) [L24] · Q7(b) calculate(amount): CFG, cyclomatic complexity, basis paths, test cases (25) [L26] · Q8(d) Depth-first top-down integration of ROOT / M1–M9 (9⅔) [L24]
2010-11 (CSE 307)	Q6(c) Verification vs validation; I/O faults for a banking module (7) [L23]

EXAM FOCUS Asked in every paper (40 rows): EP/BVA test design (2024-25, 2023-24, 2022-23, 2020-21, 2018-19), coverage test data (2022-23, 2020-21, 2018-19), white-box test cases from code (2023-24, 2012-13, 2011-12), testing types and integration (2024-25, 2022-23, 2021-22, 2019-20). Learn first: the EP/BVA table method (Lesson 27), the four coverage criteria on getTriangle (Lesson 25), and V(G) + basis paths (Lesson 26).

Lesson 23 — Testing Basics: V&V, Fault Terms, Black Box vs White Box

Slides 1–13 (Lec 11 - Software testing)

1. Quality-control techniques (sl. 1)

Fault avoidance prevents errors before release (reviews, inspections, walkthroughs, methodologies, testing, verification). **Fault tolerance** lets the system recover by itself (rollbacks, redundancy, mirroring).

2. Verification and validation (sl. 2)

	Verification	Validation
Question	Are we building the product right?	Are we building the right product?
Checks	products of a phase satisfy the conditions set at the start of that phase	the system satisfies the specified requirements (user needs)
How	testing, reviews, inspections	comparison against requirements, acceptance testing

3. Mistake, fault, failure, error (sl. 3–4)

Term	Meaning	factorial(int a) example
Mistake	human action producing an incorrect result	programmer did not consider a < 1
Fault / defect	incorrect step, process or data definition in the program	missing input validation
Failure	inability to perform the required function within the performance requirement	overflow / crash / infinite recursion
Error	difference between computed and correct value	exception or wrong output instead of an error message

```
int factorial(int a) {
    if (a == 1) return 1;          // a = 0 or a < 0 recurses forever -> stack overflow
    return a * factorial(a - 1);
}
```

4. Goals (sl. 5–7)

Testing = analysing a software item to detect differences between existing and required conditions (bugs) and to evaluate its features. Test **throughout** development. Goal: write test cases that cause failures — as few as possible, without two tests causing the same failure. Testing cannot guarantee all faults are found.

5. Black box vs white box (sl. 8–13)

	Black box	White box
Basis	specification only: inputs → outputs; internals unknown	internal structure: paths, conditions, loops, branches
Advantages	not influenced by the code; robust to implementation changes; independent testers; good for validation	finds boundaries inside the code; yields useful tests; examines data structures; good for verification

	Black box	White box
Disadvantages	misses bugs not covered by the spec (control-flow details, optimisations, alternate algorithms)	tests can share the implementation's bugs; impossible to exercise all paths (exhaustive testing explodes)

EXAM-READY ANSWER Exhaustive testing is nearly impossible (2014-15 Q2(a)): a function with two 32-bit int inputs has $2^{64} \approx 1.8 \times 10^{19}$ input pairs; at 10^9 tests per second that is about 585 years. A loop that runs up to 20 times with 5 paths per iteration has $\Sigma\ 5^k$ ($k = 1 \ldots 20$) $\approx 10^{14}$ paths. Hence we choose representative tests (EP, BVA, coverage).

Practice — Lesson 23

(a) Actual — 2017-18 Q8(a). (b) Actual — 2012-13 Q4(d) / 2010-11 Q6(c). (c) Actual — 2021-22 Q3(c). (d) Actual — 2013-14 Q3(b).

(a) Advantages and disadvantages of black-box testing; discuss performance and regression testing. (4+8)

(b) Differentiate verification and validation; defect testing and debugging. What I/O faults would you check in a banking transaction module? (6 / 7)

(c) 'It costs less to fix a bug found in the later stages of the SDLC' — agree? (5)

(d) Explain the debugging process with a diagram. (8)

✓ Solution

(a) Table §5 for black box. Performance (load) testing checks compliance with performance requirements (response time, throughput under expected load); regression testing re-runs a selected subset of tests after a change to make sure nothing that worked is broken (Lesson 24).

(b) V&V — table §2. Defect testing aims to **reveal** that a defect exists (a failing test); debugging **locates and removes** the fault that caused the failure, then re-tests. Banking I/O faults: negative or zero amount, amount with too many decimals, non-numeric input, account number of wrong length / nonexistent, withdrawal above balance or limit, wrong currency, output rounding errors, balance not updated or updated twice.

(c) Disagree. The later a bug is found, the more work products depend on it: a requirement error found in maintenance forces changes to design, code, tests and documentation and re-deployment. Studies (Boehm) give relative costs of roughly 1 (requirements) : 5 (design) : 10 (coding) : 20–50 (testing) : 100+ (after release). That is why reviews and testing start early (shift-left).

(d) Execute test cases → results compared with expected → failure observed → **suspected causes** → additional tests / instrumentation → **identified cause** → correction → regression tests. Approaches: brute force (print/dump), backtracking (from symptom back through the code), cause elimination (binary partition of the input or code).

Lesson 24 — Six Types of Testing, Integration Order, Performance Tests

Slides 14–22 (Lec 11 - Software testing)

Type	What	Who / box
Unit	individual units or small groups; code does what it is intended to at a low structural level	programmer; white box; automated
Integration	combined components tested for their interaction — units that work alone may fail together	programmers while integrating; mostly white box
Non-functional (system)	complete system vs specified requirements: stress (beyond limits), performance/load (meets performance requirements), usability (learn, input, interpret outputs)	external test group; mostly black box
Acceptance	formal test against acceptance criteria so the customer can accept or reject	customer, in their environment; black box
Regression	selective re-testing after modification: representative sample + tests most likely affected. **Smoke test** = small subset showing the system is stable and major functions work	any; re-run often
Beta	free advance version to real users who report errors. + unexpected errors, low cost, wide environments; − not systematic, low-quality reports	external users

Test plan (sl. 22): document of scope, approach, resources and schedule — test items, features, tasks, who tests, risks needing contingency. Write it early.

1. Integration strategies (textbook)

Strategy	Order	Needs
Big bang	all at once	nothing — but faults are hard to locate
Top-down	from the main module downward (depth-first or breadth-first)	**stubs** for lower modules not yet integrated
Bottom-up	from leaf modules upward in clusters	**drivers** to call the cluster
Sandwich	top-down for upper levels, bottom-up for lower	both

Stub = dummy called module that returns canned answers. **Driver** = dummy calling program that feeds inputs and checks outputs. Unit-test environment (2014-15 Q2(d)) = driver + module under test + stubs, plus test cases and results.

Internal vs external integration (2022-23 Q1(c)): internal = between components of our own system; external = between our system and third-party systems/APIs.

Practice — Lesson 24

(a) Actual — 2021-22 Q3(b). (b) Actual — 2022-23 Q1(c). (c) Actual — 2024-25 Q6(c). (d) Actual — 2013-14 Q3(a) / 2011-12 Q8(d). (e) Actual — 2019-20 Q8(b) / 2017-18 Q7(a). (f) Actual — 2024-25 Q7(a). (g) Actual — 2020-21 Q8(c). (h) Actual — 2011-12 Q7(a). (i) Actual — 2019-20 Q5(b).

(a) Briefly discuss the six types of testing. (10)

(b) Construct an example of internal and of external integration testing for a ride-sharing app. (10)

(c) Smart temperature system: Display Panel (top), Temperature Controller (middle, still under development), Sensor (low). Display and Sensor are ready. How do you integration-test? (10)

(d) Show the order of module tests for top-down integration (stubs and drivers) of a tree ROOT → M1, M2, M3; M1 → M4 → M8, M9; M2 → M5, M6; M3 → M7 (depth-first). (9⅔ / 10)

(e) Spec: 100 simultaneous users. The app failed at 90 users, then crashed at 150. Which two tests would have caught these? Also: load vs stress testing for BIIS; is code review black-box testing? (6 / 15)

(f) Course registration risks: burst to 10,000 students when registration opens; collapse when load exceeds capacity; slowdown after 8 h. For each give the performance-test type, its goal, and a JMeter configuration (threads, loop count, ramp-up, duration). (15)

(g) Smoke vs regression testing with an example. (8)

(h) Online stock-trading tests — unit, integration or system: (i) quotes within 3 s for 200 simultaneous users (ii) server accepts an order only with sufficient funds (iii) client refuses non-existent stocks (iv) order with sufficient funds is eventually executed (v) removed account cannot order. (10)

(i) ERP HRM (employment record, promotion, payroll & loan, disciplinary action, pension) interacting with Accounts; a suspended employee gets half salary, refunded if proved innocent. Give 4 inter-component and 2 inter-module integration requirements. (15)

✓ Solution

(a) One line per row of the table: purpose, who, black/white box.

(b) Internal: Ride-request module → Matching service → Fare calculator: a request from Gulshan to Banani must produce a matched driver and a fare computed with the right distance. External: our Payment module ↔ bKash/card gateway (payment success/failure callbacks), Maps API (route and ETA returned for the given coordinates), SMS gateway (OTP delivered).

(c) The missing module is in the **middle**, so use a sandwich approach with a stub and a driver: (1) Display + **stub** of the Controller that returns canned processed values ('25 °C', 'warning: 45 °C') — checks display and warnings. (2) Sensor + **driver** that plays the Controller's role: calls readTemperature() and checks the raw values/format. (3) When the Controller is ready, replace stub and driver: Sensor → Controller → Display, and re-run both sets as regression tests.

(d) Depth-first top-down: test ROOT with stubs S1, S2, S3; replace S1 by M1 (stub S4); M4 (stubs S8, S9); M8; M9; then M2 (stubs S5, S6); M5; M6; then M3 (stub S7); M7. Order of integration: ROOT, M1, M4, M8, M9, M2, M5, M6, M3, M7 — after each step re-run the earlier tests. Drivers are not needed in top-down (the real parent calls the module); they are needed only in bottom-up.

(e) Failure at 90 (< 100 users): **load (performance) testing** at the specified 100 users would have revealed it. Crash at 150: **stress testing** beyond the specified limit would show how it fails and whether it recovers gracefully. BIIS: load = 2,000 students checking results at the expected peak; stress = 10,000 at once until it breaks. Code review is **not** black-box testing — it is a static white-box technique: reviewers read the source code; nothing is executed.

Risk	Test type	Goal	JMeter (threads · ramp-up · loop · duration)
Burst to 10,000 at opening	Spike test	sudden jump in users: response time, errors, recovery	10,000 threads · ramp-up 10 s · loop 1 (or 5) · 5 min
Collapse beyond capacity	Stress test	find the breaking point and failure mode beyond capacity	step 2,000 → 20,000 threads (Stepping/Ultimate Thread Group) · ramp-up 10 min · loop forever · 30 min
Slowdown after 8 h	Endurance (soak) test	memory leaks, resource exhaustion under steady load	1,500 threads · ramp-up 5 min · loop forever · 8–10 h

(g) Smoke: a small, fast subset run on every new build to decide whether it is stable enough to test (login, search, add to cart, checkout open). Regression: a larger selection re-run after a change to make sure existing features still work (after changing the discount code, re-run all cart, pricing and checkout tests). Smoke is wide and shallow; regression is deep on what the change may affect.

(h) (i) system (performance with the whole system) · (ii) unit test of the ordering server's validation, then integration with the account database · (iii) unit test of the client (input validation), and integration if the stock list comes from the server · (iv) integration (order server + accounts + external price server) and system · (v) integration (account DB ↔ ordering server) and system.

(i) Inter-component: (1) Disciplinary Action → Payroll: suspension sets salary to 50 % from the suspension date; (2) Disciplinary Action → Payroll: proved innocent → withheld 50 % paid back as arrears; (3) Promotion → Employment Record & Payroll: new grade updates the record and the salary scale; (4) Payroll ↔ Loan: monthly loan instalment deducted from salary, loan balance updated; (also Employment Record → Pension: service length). Inter-module: (1) Payroll → Accounts: monthly salary journal posted (expense, bank) including half-salary during suspension and later arrears; (2) Pension / Loan → Accounts: pension and loan disbursements and recoveries posted to the right ledgers.

Lesson 25 — Code Coverage Criteria

Slides 1–8 (1 - Coverage.pptx)

Code coverage = degree to which the source code is executed by a test suite. High coverage → more thoroughly tested → lower chance of hidden bugs (sl. 1).

Criterion	Question (sl. 2)
Method coverage	Have all methods been called?
Statement coverage	Have all statements been executed?
Decision / branch coverage	Has every decision taken both its true and false outcome?
Condition coverage	Has every individual condition (atomic boolean) been both true and false?

Worked example — getTriangle (sl. 3–6)

```
String getTriangle(int a, int b, int c) {
  D1: if (a == b && b == c) { success(); return "Equilateral"; }     // conditions C1: a==b, C2: b==c
  D2: if (a == b || b == c || c == a) return "Isosceles";           // C3: a==b, C4: b==c, C5: c==a
      return "Scalene";
}
```

Test (a,b,c)	C1	C2	D1	C3	C4	C5	D2	Result
(1,1,2)	T	F	F	T	–	–	T	Isosceles
(2,2,2)	T	T	T					Equilateral
(2,2,3)	T	F	F	T	–	–	T	Isosceles
(1,2,2)	F	–	F	F	T	–	T	Isosceles
(2,1,2)	F	–	F	F	F	T	T	Isosceles
(2,3,4)	F	–	F	F	F	F	F	Scalene

- **Method** (sl. 3): (1,1,2) and (2,2,2) — getTriangle and success() both called.
- **Condition** (sl. 4): (2,2,3), (2,2,2), (1,2,2), (2,1,2) — every C1…C5 is T at least once and F at least once (C5 is never F here under short-circuit; the slide counts C3–C5 by whether each was T/F when evaluated).
- **Branch** (sl. 5) and **statement** (sl. 6): add (2,3,4) so that D2 is false and 'return "Scalene"' runs.
- Hierarchy: branch coverage ⇒ statement coverage; condition coverage does not imply branch coverage (combine them: condition/decision coverage).

Benefits, limitations, tools (sl. 7–8)

Benefit: measures completeness of tests and shows missed paths. Limits: high coverage ≠ correctness; tools only see the code that exists, not missing requirements; do not write tests just to please the tool. Tools: djUnit, EclEmma (instruction, block, line, method, type coverage), Clover.

Practice — Lesson 25

(a) Actual — 2022-23 Q4(c). (b) Actual — 2020-21 Q6(b) / 2018-19 Q2(b). (c) Actual — 2023-24 Q5(c).

(a) something(int x) { if (x < 0 || y < x) { y := -y; x := -x; } z := x; } — show a minimal-length data set that achieves condition coverage. (8)

(b) Give an example test execution ensuring method, branch and condition coverage. What are the characteristics of good unit testing? (15)

(c) Why is high test coverage important? What risks arise if coverage is too low? (8)

✓ Solution

(a) Conditions: A = (x < 0), B = (y < x); y is an outer (global) variable, so it is part of the test data. Condition coverage: A and B must each be true once and false once.

Test	x	y	A: x<0	B: y<x	Decision
T1	-1	0	T	F (0 < -1 false)	T
T2	1	0	F	T (0 < 1)	T

Two tests achieve condition coverage if both operands are evaluated. Under short-circuit '||', B is not evaluated in T1, so add T3: x = 1, y = 5 (A = F, B = F, decision F) — 3 tests, which also give decision coverage. Minimal answer: {(−1, 0), (1, 0)} without short-circuit; {(−1, 0), (1, 0), (1, 5)} with it.

(b) Use getTriangle: method coverage {(1,1,2), (2,2,2)}; condition coverage {(2,2,3), (2,2,2), (1,2,2), (2,1,2)}; branch coverage = those + (2,3,4). Good unit tests (IJ Mam's Unit Testing deck): **Automatic** (no human input), **Atomic** (decides pass/fail itself), **Single responsibility** (one feature), **Independent** (runs in isolation), **Repeatable** (same result every run); also fast and with descriptive names.

(c) Importance: shows how much of the code the tests exercise; reveals untested paths, dead code and missing tests; gives confidence for refactoring and regression. Low coverage risks: bugs hidden in unexecuted branches (error handling, rare conditions) reach production; changes break untested code silently; security holes in unchecked input paths; poor estimate of quality. But 100 % coverage still cannot find missing requirements.

Lesson 26 — White-Box Test Design: Basis Paths and Loop Testing

Textbook (Pressman ch. 23) — needed for 2023-24, 2014-15, 2013-14, 2012-13, 2011-12

1. Basis path testing

1. Draw the flow graph: nodes = statements/decision points, edges = control flow; a compound condition becomes one node per simple condition.
2. Cyclomatic complexity V(G) = E − N + 2 = P + 1 (P = number of simple decisions) = number of regions.
3. List V(G) linearly independent paths (each adds at least one new edge).
4. Design one test case per path (input + expected output).

$$V(G) = E - N + 2 = P + 1$$

2. Loop testing

Loop type	Tests (n = max passes)
Simple loop	skip the loop; 1 pass; 2 passes; m passes (m < n); n − 1, n, n + 1 passes
Nested loops	start at the innermost loop with outer loops at minimum; test it as a simple loop; work outward
Concatenated	independent → as simple loops; dependent → as nested

Practice — Lesson 26

(a) Actual — 2023-24 Q3(b). (b) Actual — 2011-12 Q7(b). (c) Actual — 2012-13 Q4(c). (d) Actual — 2014-15 Q2(b) / 2013-14 Q3(d).

(a) Generate white-box test cases for FindMean(FILE ScoreFile): read scores until EOF, add positive scores, count them; print the mean if count > 0 else 'No scores found in file'. (10)

(b) calculate(int amount): rushCharge = 14.50 if nextday is 'yes'; tax = amount × .0725; shipping by amount ≥ 1000 (×.06), ≥ 200 (×.08), ≥ 100 (13.25), ≥ 50 (9.95), ≥ 25 (7.25), else 5.25, each + rushCharge; total = amount + tax + ship. Draw the CFG, compute cyclomatic complexity, list the basis paths and their test cases. (25)

(c) countingSort(ar, n, k): if (n > 1) { B = calloc(k); for i < n: B[ar[i]]++; for i < k: while (B[i]-- > 0) ar[idx++] = i; free(B); } return ar. Flow graph, independent paths, test cases. (20)

(d) Loop-testing test cases for a module that finds the maximum of (sums) n numbers. (12 / 10)

✓ Solution

(a) Decisions: while (!EOF), if (Score > 0), if (count > 0) → V(G) = 3 + 1 = **4**.

```
Flow graph:  1 init + Read -> 2 while(!EOF)? --F--> 6 if(count>0)? --T--> 7 mean, print --> 9 end
                                --T--> 3 if(Score>0)? --T--> 4 sum+=, count++ --> 5 Read --> 2
                                                      --F-----------------------> 5      6 --F--> 8
'No scores' --> 9
```

Path	Nodes	Test data (file)	Expected
P1	1-2-6-8-9	empty file	No scores found in file
P2	1-2-3-4-5-2-6-7-9	80	The mean score is 80.000000
P3	1-2-3-5-2-6-8-9	-5 (or 0)	No scores found in file

Path	Nodes	Test data (file)	Expected
P4	1-2-3-5-2-3-4-5-2-6-7-9	-5, 60	mean 60 (negative ignored)

(b) Decisions: nextday == 'yes' (1) + five 'if' tests in the chain (≥1000, ≥200, ≥100, ≥50, ≥25) = 6 → V(G) = 6 + 1 = **7** (check: N = 15, E = 20 for the graph below → 20 − 15 + 2 = 7).

```
1 rush=0; if nextday=="yes" --T--> 2 rush=14.50 --> 3
                          --F-----------------------> 3 tax=amount*.0725; if amount>=1000
3 --T--> 4 ship=amount*.06+rush ----------------------------------------> 14
3 --F--> 5 if amount>=200 --T--> 6 ship=amount*.08+rush -------------------> 14
         5 --F--> 7 if amount>=100 --T--> 8 ship=13.25+rush ---------------> 14
                  7 --F--> 9 if amount>=50 --T--> 10 ship=9.95+rush -------> 14
                           9 --F--> 11 if amount>=25 --T--> 12 ship=7.25+rush -> 14
                                    11 --F--> 13 ship=5.25+rush -------------> 14
14 total=amount+tax+ship; return   (end)        N = 15 incl. end node, E = 20
```

Path	Route	nextday, amount	Expected total
1	1-3-4-14	no, 1000	1000 + 72.50 + 60.00 = 1132.50
2	1-2-3-4-14	yes, 1000	1000 + 72.50 + 74.50 = 1147.00
3	1-3-5-6-14	no, 200	200 + 14.50 + 16.00 = 230.50
4	1-3-5-7-8-14	no, 100	100 + 7.25 + 13.25 = 120.50
5	1-3-5-7-9-10-14	no, 50	50 + 3.625 + 9.95 = 63.575
6	1-3-5-7-9-11-12-14	no, 25	25 + 1.8125 + 7.25 = 34.0625
7	1-3-5-7-9-11-13-14	no, 10	10 + 0.725 + 5.25 = 15.975

(c) Decisions: n > 1, for-1, for-2, while → V(G) = 4 + 1 = **5**.

```
1 i, idx=0 -> 2 if(n>1)? --F--> 10 return ar
2 --T--> 3 B=calloc(k) -> 4 for i<n? --T--> 5 B[ar[i]]++ --> 4
                           4 --F--> 6 for i<k? --T--> 7 while(B[i]-- > 0)? --T--> 8 ar[idx++]=i --> 7
                                                 7 --F--> 6 (next i)
                                    6 --F--> 9 free(B) --> 10
```

Path	Route	Test (ar, n, k)	Expected
P1	1-2-10	[5], n = 1, k = 6	[5] unchanged (no sorting for n ≤ 1)
P2	1-2-3-4-6-9-10	infeasible: n > 1 forces loop 1 to run; k = 0 would also be invalid	—
P3	1-2-3-4-5-4-6-9-10	infeasible with n > 1 and k ≥ 1 (loop 2 must run)	—
P4	1-2-3-4-5-4-6-7-6-9-10 (+ while false)	[0,0], n = 2, k = 2 → for i = 1 B[1] = 0	[0, 0]
P5	… 6-7-8-7-6 …-9-10	[2,0,1], n = 3, k = 3	[0, 1, 2]

Note in the answer that some basis paths are infeasible because n > 1 forces both loops; cover their edges with the feasible paths P4–P5. Add a duplicate-value test [1,1,0] (n = 3, k = 2) → [0,1,1] to exercise the while loop more than once.

(d) Simple loop over n numbers (max N allowed, say 100):

Case	n	Data (max)	Expected
Skip loop	0	[]	error / defined value (no max)
One pass	1	[7]	7
Two passes	2	[3, 9]	9
Typical m	5	[4, -2, 11, 6, 0]	11
N − 1	99	values, largest last	largest
N	100	largest first	largest

Case	n	Data (max)	Expected
N + 1	101	—	rejected / overflow handled

For the sum module, the same seven cases with expected sums (0 for n = 0, 7, 12, 19, …).

Lesson 27 — Equivalence Partitioning, BVA, Positive and Negative Testing

Slides 1–14 (2 - Equivalence partitioning.pptx)

1. Equivalence partitioning (sl. 1–4)

Divide the input domain into partitions whose values the software treats the same way; test one value per partition. If one value works, assume all in the partition work; if it fails, assume all fail.

Partition	Balance	Interest
1 invalid	< 0	undefined → exception
2	0 ≤ B < 1000	0.5 %
3	1000 ≤ B < 10000	0.75 %
4	10000 ≤ B	1.0 %

```
// sl. 3 — how NOT to test: 0, 500, 1000, 2000, 5000 ... many values, only partitions 2 and 3 covered
// sl. 4 — one value per partition
try { calculateSavingsInterest(-1.0); fail("Exception expected"); } catch (Exception ex) { }
assertEquals(0.50,  calculateSavingsInterest(100.0));     // 100 x 0.5 %
assertEquals(15.0,  calculateSavingsInterest(2000.0));    // 2000 x 0.75 %
assertEquals(200.0, calculateSavingsInterest(20000.0));   // 20000 x 1 %
```

2. Boundary value analysis (sl. 5)

Most errors occur at partition boundaries, so test on and next to each boundary: −0.01 | 0, 0.01, 999.99 | 1000, 9999.99 | 10000.

> **MEMORIZE** BVA for a closed range [a, b] of integers: a − 1, a, a + 1, (nominal), b − 1, b, b + 1. **Weak ECT** = enough tests that every class of every input appears once → max(classes per input). **Strong ECT** = every combination → product of the class counts.

3. Positive and negative testing (sl. 6–14)

Positive testing: valid input, the application must work and must not show an error (phone number 6184536032). Negative testing: invalid input, the application must handle it gracefully (xyZ#53()032). Typical negative scenarios:

- required fields left empty (sl. 8)
- data of the wrong type for the field (sl. 9)
- more characters than allowed (sl. 10)
- values outside the allowed bounds — day −1, month 13 (sl. 11)
- values beyond the data type range — int −32,768…32,767 (2 bytes), ±2,147,483,647 (4 bytes), float ±3.4E+38 (sl. 12)
- unreasonable data — 300-character name, age 200 (sl. 13)
- web session — open URLs recorded while logged in after logging out (sl. 14)

Practice — Lesson 27

(a) Actual — 2022-23 Q1(b). (b) Actual — 2024-25 Q7(b). (c) Actual — 2024-25 Q5(a). (d) Actual — 2023-24 Q1(c). (e) Actual — 2023-24 Q3(c) / 2013-14 Q3(e). (f) Actual — 2020-21 Q7(a) / 2018-19 Q3(a). (g) Actual — 2021-22 Q4(c). (h) Actual — 2020-21 Q5(b). (i) Actual — 2018-19 Q1(b).

(a) Test cases for income tax: 0 % below 4 lac, 10 % for 4–10 lac, 20 % above 10 lac. (10)

(b) Irrigation pump: OFF if M < 30, LOW if 30 ≤ M ≤ 70, HIGH if M > 70 (M integer). All BVA tests for the LOW partition. (10)

(c) Lab-section registration with independent inputs: credits C1 < 60, C2 60–99, C3 > 99, C4 invalid; CGPA G1 < 2.50, G2 2.50–3.49, G3 ≥ 3.50, G4 invalid; status S1 not approved, S2 approved, S3 invalid/missing. (i) minimal weak ECT suite (ii) number of strong ECT cases (iii) a defect the weak suite may miss. (15)

(d) Design test cases for your password policy (Mentcare). (10)

(e) Explain the principle of equivalence-partition testing. Why is it used in real software? (8 / 6⅔)

(f) GM entitled to one laptop (and two mobile phones) per year via a requisition app: positive and negative tests. (10 / 7)

(g) Five negative-testing ideas for a web application, with examples. (5)

(h) EP: test input set for whether the Release Order cost (size × rate × (1 − discount)) is computed properly. (12)

(i) EP: test input set for different user roles of BIIS. (12)

✓ Solution

(a) Income I in taka (assume 4–10 lac includes both ends: 4,00,000 ≤ I ≤ 10,00,000).

ID	Class / boundary	Income (Tk)	Expected tax
T1	invalid: negative	−1	error
T2	0 % class (nominal)	2,00,000	0
T3	lower bound 0 %	0	0
T4	just below 4 lac	3,99,999	0
T5	4 lac (10 % starts)	4,00,000	40,000
T6	10 % nominal	7,00,000	70,000
T7	10 lac (last 10 %)	10,00,000	1,00,000
T8	just above 10 lac	10,00,001	2,00,000.20
T9	20 % nominal	15,00,000	3,00,000
T10	invalid type	'abc' / empty	validation error

(If the tax is slab-wise progressive, expected values change, e.g. 7 lac → 10 % of 3 lac = 30,000; state your assumption.)

(b) LOW partition 30 ≤ M ≤ 70. Boundary tests: 29 → OFF, **30** → **LOW**, **31** → **LOW**, **50** → **LOW** (nominal), **69** → **LOW**, **70** → **LOW**, 71 → HIGH. The two outside values check the neighbouring partitions; a robust set adds non-integer / missing reading → error.

(c)(i) Weak ECT: every class of every input at least once; the largest input has 4 classes → **4 tests**.

Test	Credits	CGPA	Status	Expected
W1	C1: 45	G1: 2.20	S1 not approved	reject
W2	C2: 80	G2: 3.00	S2 approved	allowed (if rule permits)
W3	C3: 120	G3: 3.75	S3 missing	error: invalid status
W4	C4: −5 / 'abc'	G4: 4.80	S2 approved	error: invalid input

(ii) Strong ECT = all combinations = 4 × 4 × 3 = **48** test cases.

(iii) Defects that depend on a combination are missed, e.g. the rule 'credits 60–99 AND CGPA ≥ 3.50 AND approved → allowed' coded with the wrong operator (< 3.50) for C2 only. Weak tests use C2 only with G2, so the C2 + G3 + S2 combination is never run and the bug escapes; similarly an invalid CGPA (G4) together with S1 might crash instead of reporting an error.

(d) Assumed policy: length 8–16; at least one upper-case, lower-case, digit and symbol; not containing the username or a dictionary word; lock after 5 failed attempts; expiry 90 days; no reuse of the last 5.

ID	Input / action	Expected
P1	'Ment@2025x' (10 chars, all classes)	accepted (positive)
P2	'Ab1@xyz' (7 chars) / 'Ab1@xyzw' (8) / 16 / 17 chars	reject / accept / accept / reject (BVA)
P3	'mentcare1@' (no upper case)	rejected
P4	'MENTCARE@1' (no lower)	rejected
P5	'Mentcare@@' (no digit)	rejected
P6	'Mentcare12' (no symbol)	rejected
P7	contains username 'nurse_rina'	rejected
P8	child's name / dictionary word 'Rahim2018!'	rejected
P9	5 wrong logins	account locked, admin alerted
P10	reuse one of last 5 passwords	rejected
P11	password older than 90 days	forced change at login
P12	empty password	rejected

(e) Principle — §1. Why used: exhaustive testing is impossible; EP reduces the number of tests to one per class while still covering every kind of behaviour, avoids redundant tests (sl. 3), and makes test design systematic and explainable; combined with BVA it targets where faults cluster.

(f) Positive: GM requests the first laptop of the year → approved; GM requests 1st and 2nd phone → approved; request in a new year after last year's laptop → approved. Negative: second laptop in the same year → rejected with message; third phone → rejected; non-GM employee requests → rejected; quantity 0 / −1 / 'abc' → validation error; required fields (model, justification) empty → error; date in the future/past year → error.

(g) Any five of sl. 8–14 with an example each: empty required field (submit without email); wrong type (letters in phone); too many characters (300-char name); out of bounds (day 32, month 13); beyond data-type range (quantity 3,000,000,000); unreasonable data (age 200); access URLs after logout; plus SQL/script injection in a text box ('; DROP TABLE …).

(h) Inputs: size (column-inch), rate, discount %. Partitions: size invalid ≤ 0 | valid 1–max page | > page size; rate invalid ≤ 0 | valid; discount invalid < 0 | 0 | 0–100 | > 100; date with rate change.

T	Size	Rate	Discount	Expected cost
1	10	500	10 %	10 × 500 × 0.9 = 4,500
2	10	500	0 %	5,000
3	0 / −2	500	10 %	error
4	10	0 / −1	10 %	error
5	10	500	−5 % / 120 %	error
6	10	500	100 %	0 (boundary)
7	1 / max page size	500	10 %	450 / max × 450
8	10	rate changed on publish date	10 %	uses the rate valid on that date

(i) Partitions of the role input: Student, Teacher/Advisor, Head of department, Register office/Admin, Guest (not logged in), invalid/expired role. One test per class: e.g. student can view own result but not edit (positive) and gets 'access denied' on the grade-entry page (negative); advisor can approve registration of advisees only; admin can open all; guest is redirected to login; a tampered role token is rejected.

Quick Notes — Software Testing

Revision points for Lessons 23–27.

L23 · Basics

- Verification = built right; validation = right product.
- Mistake → fault → failure; error = computed − correct.
- Black box: spec, independent, misses code paths; white box: paths, can share bugs.

L24 · Types

- Unit, integration, non-functional (stress, performance/load, usability), acceptance, regression (smoke), beta.
- Top-down needs stubs; bottom-up needs drivers; missing middle → stub + driver.
- Spike (burst), stress (beyond capacity), endurance/soak (hours).

L25 · Coverage

- Method, statement, branch (decision), condition. getTriangle: (1,1,2),(2,2,2) / +(2,2,3),(1,2,2),(2,1,2) / +(2,3,4).
- x<0 || y<x: (−1,0),(1,0) [+ (1,5) with short-circuit].

L26 · Basis paths

- V(G) = E − N + 2 = P + 1. FindMean 4; calculate() 7; countingSort 5.
- Loop tests: 0, 1, 2, m, n−1, n, n+1.

L27 · EP / BVA

- One test per partition; BVA a−1, a, a+1, b−1, b, b+1.
- Weak ECT = max classes (4); strong = product (4×4×3 = 48).
- Negative tests: empty, wrong type, too long, out of bounds, type range, unreasonable, session.